Density Dependent Marked Point Process Framework (DenMark) Tutorial

Mingchi Xu, Alexandra Schmidt** and Qihuang Zhang*

This tutorial shows how to use **DenMark** to quantify the correlation between gene expression and cell density. We also show how to identify the density-correlated genes (DCG) in the single-cell resolution spatial transcriptomics (ST) dataset.

We use the 10X Xenium breast cancer dataset as an example **(dataset can be found here)**. Section 1 describes data loading and ROI selection, followed by coordinate system normalization and data preprocessing in Section 2. Section 3 presents the quantification of the relationship between gene expression and cell density, while Section 4 focuses on the identification of DCGs.

We start with two files:

- A 2D spatial locations of cells worksheet (cell ID $\times$ 2D coordinate);
- A raw gene expression over the cells worksheet (cell ID $\times$ gene expression).

All the content in this tutorial is run by R-4.3.2 under a PC with Windows 11 operation.

Prepare packages and functions

DenMark is built on a stan environment, so a stan installation environment is needed. This include the installation of *stan*, *rstan*, and *cmdstanr*.

```
# [Code Block 0.1] Installation of rstan, and cmdstanr

#  ---- Dependent on the operation, the installation might be different ----
# Please refer to the installation page:  https://mc-stan.org/install

rm(list=c())
# Finally, you are supposed to load the packages:
library('rstan')
library('cmdstanr')
```

Some packages are also useful for the data analysis, make sure you have downloaded them before we get started.

```
# [Code Block 0.2] Load Other useful packages

# load other packages for visualization and DenMark analysis
```

```r
# (a) For data analysis:
library(reshape2)
library(dplyr)
library('MASS')
library(fields)
library(Matrix)
library(tidyverse)
library(magrittr)
library(tidybayes)
library(coda)
library(loo)

# (b) For visualization:
library('ggplot2')
library(cowplot)
library(viridis)
library(rethinking)
library(ggridges)
library(ggrepel)

# (c) Self-built posterior summary functions:
q2.5 <- function(x) quantile(x, prob = 0.025)
q25 <- function(x) quantile(x, prob = 0.25)
q50 <- function(x) quantile(x, prob = 0.50)
q75 <- function(x) quantile(x, prob = 0.75)
q97.5 <- function(x) quantile(x, prob = 0.975)

# (d) Matern32 kernel: covariance function for Matern (nu = 3/2)
matern32 <- function(d, sigma, rho) {
  sigma^2*(1 + sqrt(3) * d / rho) * exp(-sqrt(3) * d / rho)
}

# (e) cmdstanr settings for showing more rows
options(cmdstanr_max_rows = 20)

# Set the workplace
setwd('C:/Users/xumc7/OneDrive/Desktop/Tutorial')

# (f) DenMark functions and stan file directory:
source('DenMark/DenMark-functions.R')
denmark_stan       <- 'DenMark/DenMark-M2.stan'
denmark_simple_stan <- 'DenMark/DenMark-M1.stan'
```

1. Load single-cell resolution ST data, select hyperparameters

Now load the two matrices. We use the **In Situ Sample 1, Replicate 1** as an example.

```r
# [Code Block 1.1] Load two matrices

# (a) Cell ID * Gene expression:
cell_gene <- read.csv('cell_by_gene_expression.csv')
# (b) Cell ID * 2D spatial locations:
```

```r
cell_spatial <- read.csv('cell_spatial.csv')

# check the dimensions:
dim(cell_gene) # cell counts * gene numbers
```

```
## [1] 167780    322
```

```r
# check the gene names:
colnames(cell_gene)[2:6] # the first 5 genes in the dataset
```

```
## [1] "ABCC11" "ACTA2"  "ACTG2"  "ADAM9"  "ADGRE5"
```

Then hyper parameters are user-defined: the grid number (controls the grid approach approximation precision), the number of basis functions at the x-/y- axis and the boundary factor

```r
# [Code Block 1.2] Select hyper-parameters

# Hyperparameters that influence the approximate precision:
grid_size<- 10      # grid cells at the x-/y- direction
basis_choice <- 25  # number of basis functions for the HSGP. higher number makes a higher HSGP approxi
boundary_choice <- 1.5 # range from [1, Inf); boundary factor for HSGP to avoid boundary effect
```

2. ROI Selection, Coordinate Normalization and Data Preprocessing

The ROI is chosen as the research interest needs. For the HSGP to be well-defined, we need to move the ROI under the original coordinates to an ROI symmetric about the origin.

In this section we describe running DenMark over a global ROI (the whole tissue) for the large range analysis.

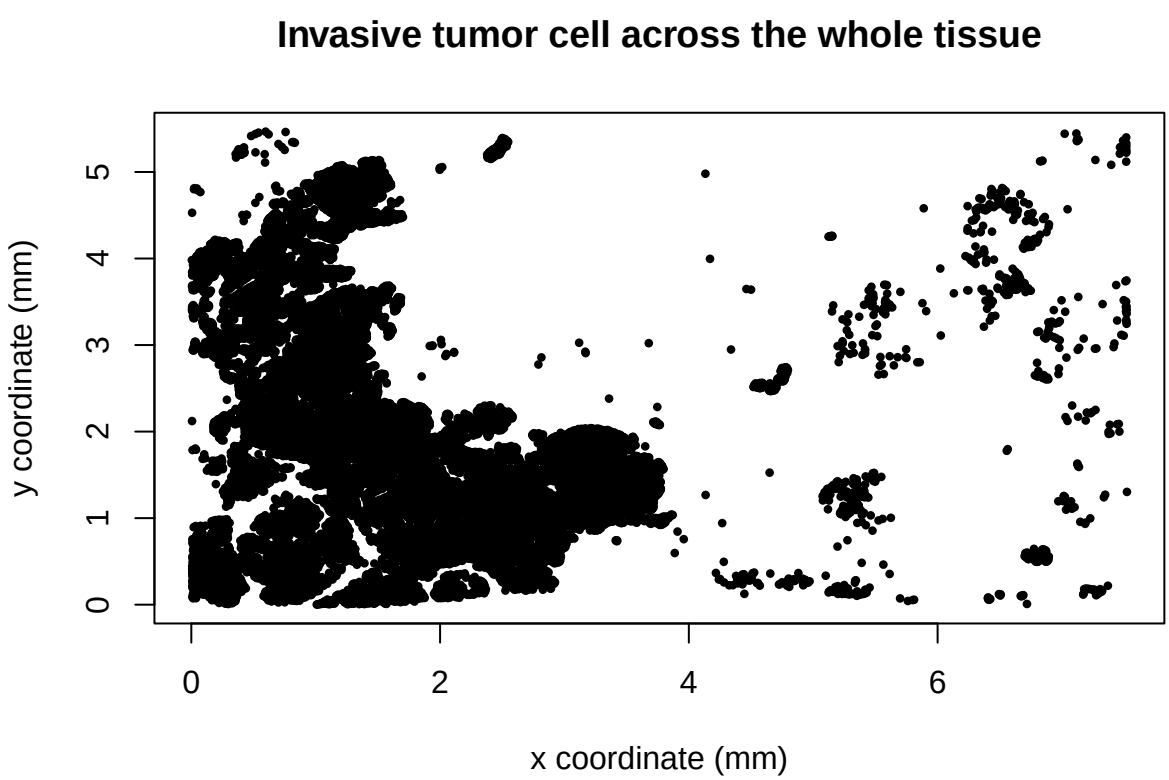
```r
#  [Code Block 2.1] Scale the coordinates into mm*mm coordinate system
cell_spatial$x<- cell_spatial$x/1000
cell_spatial$y<- cell_spatial$y/1000

# check the dataset:
head(cell_spatial)
```

```
##   X         x         y       celltype
## 1 0 0.8462731 0.3275442         DCIS_2
## 2 1 0.8253991 0.3295058         DCIS_2
## 3 2 0.8484960 0.3314183      Unlabeled
## 4 3 0.8242057 0.3321048 Invasive_Tumor
## 5 4 0.8431182 0.3315164         DCIS_2
## 6 5 0.8486596 0.3347365      Unlabeled
```

We select the invasive tumor cell across the whole tissue as an example.

```r
# [Code Block 2b.1] Extract the invasive tumor and visualize

# extract the cell type:
center_slice_cell_location_tumor <- cell_spatial[which(cell_spatial$celltype=='Invasive_Tumor'),]
```

```r
# extract the corresponding genes at the same locations
gene_tumor <- cell_gene[which(cell_spatial$celltype=='Invasive_Tumor'),]

# visualize the cell locations: each black dot is a cell location
plot(center_slice_cell_location_tumor$x,
     center_slice_cell_location_tumor$y,
     main = 'Invasive tumor cell across the whole tissue',
     xlab='x coordinate (mm)',
     ylab='y coordinate (mm)',
     pch = 16,
     cex = 0.6)
```




```r
# Note: for the short range analysis, we can also do similar procedure
# to extract cell locations and gene expression

# e.g. selected invasive tumor ROI by pathologist: [0.5, 1.5]*[2, 3]
#center_slice_cell_location_invasive <- cell_spatial[which(cell_spatial$x>=0.5 & cell_spatial$x<= 1.5 &

# corresponding gene expression at the ROI
#gene_invasive<- cell_gene[which(cell_spatial$x>=0.5 & cell_spatial$x<= 1.5 & cell_spatial$y>= 2 & cell_
```

```r
# [Code Block 2b.2] Quality control
# filter out the genes with (1) the lowest 5% , (2) zeros with more than 5% locations
gene_tumor <- gene_tumor[,-1]
```

```r
# Convert to matrix (if not already)
mat <- as.matrix(gene_tumor)

# (1) Compute variance per gene
gene_var <- apply(mat, 2, var)

# Keep top 95% high-variance genes
var_threshold <- quantile(gene_var, 0.05)
genes_high_var <- names(gene_var[gene_var >= var_threshold])

# (2) Non-zero filter (>=5%)
nonzero_fraction <- colSums(mat > 0) / nrow(mat)
genes_nonzero <- names(nonzero_fraction[nonzero_fraction >= 0.05])

# Combine both filters
genes_keep <- intersect(genes_high_var, genes_nonzero)

# Filtered dataset
gene_tumor_filtered <- mat[, genes_keep, drop = FALSE]

# Check the number of filtered genes
# length(colnames(gene_tumor_filtered)) # 174 filtered genes

# Check the first 10 filtered genes
colnames(gene_tumor_filtered)[1:10]
```

```
##  [1] "ABCC11"   "ACTA2"    "ACTG2"    "ADAM9"    "AGR3"     "AIF1"
##  [7] "ANKRD28"  "ANKRD29"  "ANKRD30A" "APOBEC3B"
```

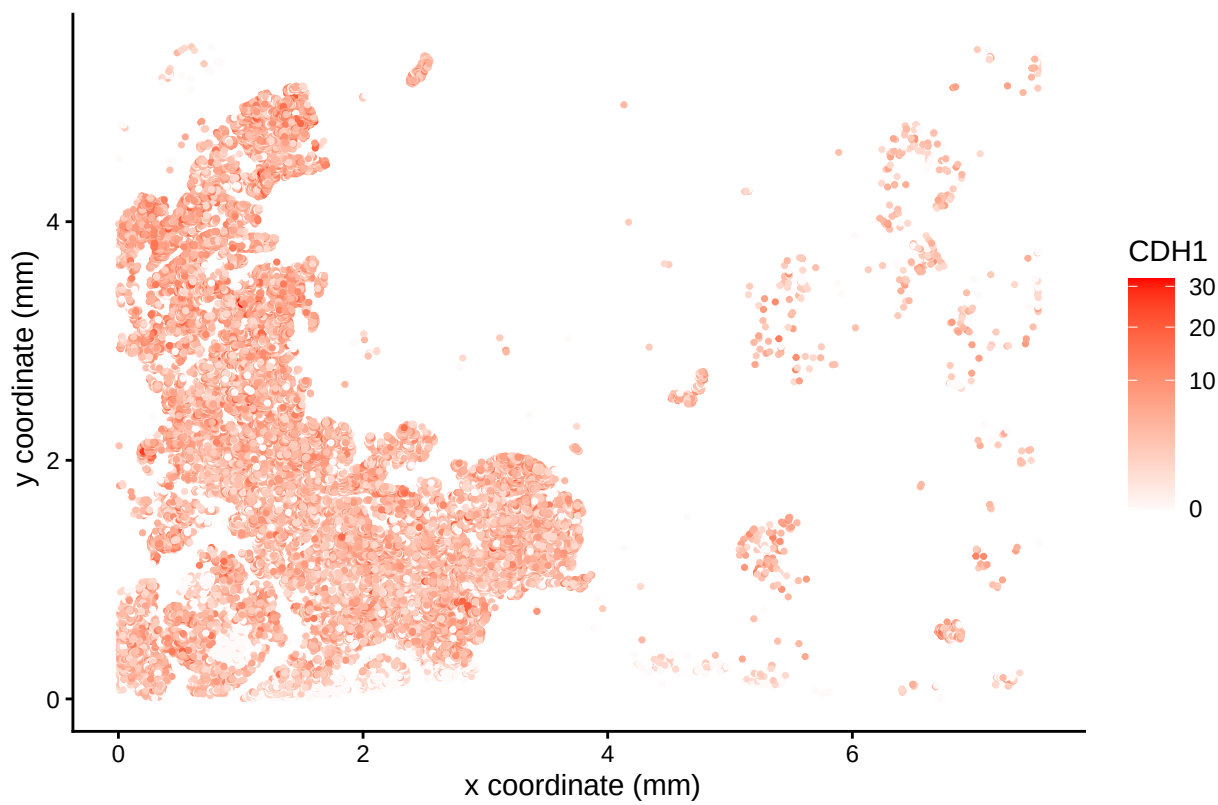
The combined filtered dataset is defined.

```r
# [Code Block 2b.3] Rearrange the dataset
final_large <- as.data.frame(cbind(center_slice_cell_location_tumor$x, # x-coordinate
                                   center_slice_cell_location_tumor$y, # y-coordinate
                                   gene_tumor_filtered # filtered gene expression
                                   ))

# rename the first two columns
colnames(final_large)[1:2]<- c('x','y')

# check the first few columns
head(final_large[,c(1:5)])
```

```
##            x         y ABCC11 ACTA2 ACTG2
## 4  0.8242057 0.3321048      0     0     0
## 19 0.8404538 0.3750625      0     1     0
## 20 0.8342422 0.3784625      0     0     0
## 22 0.8352884 0.3846740      0     0     3
## 23 0.8241894 0.4106644      0     0     0
## 35 0.8353702 0.4604058      0     0     0
```

Here, we are going to show the analysis for one typical gene expression, say, gene $CDH1$.

```r
# [Code Block 2b.4] Select your typical gene for the analysis

# Please refer to the selected informative gene list for selecting your gene of interest:
#  colnames(final_large)[-c(1,2)]

gene_choice <- 'CDH1'
df_large_select <- data.frame(final_large$x,
                              final_large$y,
                              final_large$CDH1)

colnames(df_large_select) <- c('x', 'y', gene_choice)


# we can visualize the gene expression pattern at the point pattern over the region:
ggplot(df_large_select, aes(x = x, y = y, color = CDH1)) +
  geom_point(size = 0.6, alpha = 1) +
  scale_color_gradient(low = "snow1", high = "red", trans = "sqrt") +
  labs(title="", x=" x coordinate (mm)", y=" y coordinate (mm)")+
  theme(text = element_text(size=20),
        strip.text.x = element_text(size=20),
        strip.text.y = element_text(size=20))+
  theme(legend.text = element_text(size=50))+
  theme_classic()
```



```r
# [Code Block 2b.5] Normalization of the locations

# the ROI is defined as [-L1, L1]*[-L2, L2].
# The half range L1 and L2 are transformed from the original coordinate system
L1<- max(abs(center_slice_cell_location_tumor$x-
               (min(center_slice_cell_location_tumor$x)+
                  max(center_slice_cell_location_tumor$x))/2 ))

L2<- max(abs(center_slice_cell_location_tumor$y-
               (min(center_slice_cell_location_tumor$y)+
                  max(center_slice_cell_location_tumor$y))/2 ))

# prepare the range parameter:
xRangeDat <- c(-L1,L1)
yRangeDat <- c(-L2,L2)
```

We then prepare the other input elements before running DenMark:

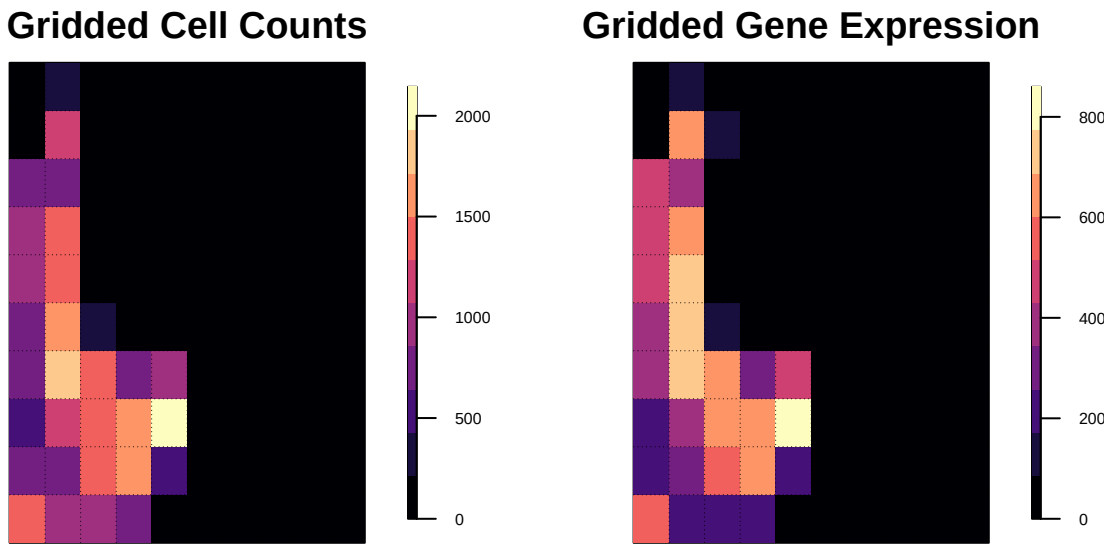
```r
# [Code Block 2b.5] Calculate the input of DenMark

# Use functions 'buildgridpp' and 'grid_pts_centroid' from DenMark-function.R
buildgrid_result <- buildgridpp(grid_size = grid_size,
                                dataset = df_large_select)

# Note:
# buildgrid_result[[1]]: cell counts + gene expression counts at the 100 grids
# buildgrid_result[[2]]: matrix form for the cell counts
# buildgrid_result[[3]]: matrix form for the gene expression counts



grid_centroid_result <- grid_pts_centroid(grid_size = grid_size,
                                          dataset = df_large_select)

# Note:
# grid_centroid_result[[1]]: locations of the grids
# grid_centroid_result[[2]]: grid area
# grid_centroid_result[[3]]: the total number of grids
```

3. Univarite Gene Analysis: Quantify the density-expression association

Now we will apply the grid approach to analyze the discretized cell locations and gene expression. First, let's visualize those grid-level cell counts and gene expression.

```r
# [Code Block 3.5] Check the grid visualization for the discretized data

# load the gridded cell counts and gene expression
loc_mat <- buildgrid_result[[2]]
marks_mat<- buildgrid_result[[3]]

# Helper to make breaks
make_breaks <- function(mat, n_colors = grid_size) {
```

```r
  seq(min(mat, na.rm = TRUE),
      max(mat, na.rm = TRUE),
      length.out = n_colors + 1)
}


#  Cell Counts | Gene expression
layout(matrix(1:2, nrow = 1), widths = c(1, 1), respect = F)

# Points
par(mar = c(8, 4, 2, 3))
image.plot(
  loc_mat,
  breaks = make_breaks(loc_mat, grid_size),
  col = viridis(grid_size, option= 'magma'),
  main = "Gridded Cell Counts",
  xlab = "",
  ylab = "",
  useRaster = FALSE,
  legend.width = 0.6,
  legend.mar = 2,
  xaxt = "n",
  yaxt = "n",
  axis.args = list(cex.axis = 0.5)
)
grid(nx = ncol(loc_mat), ny = nrow(loc_mat), col = "black", lwd = 0.5)

# Marks
par(mar = c(8, 4, 2, 3))
image.plot(
  marks_mat,
  breaks = make_breaks(marks_mat, grid_size),
  col = viridis(grid_size, option= 'magma'),
  main = "Gridded Gene Expression",
  xlab = "",
  ylab = '',
  useRaster = FALSE,
  legend.width = 0.6,
  legend.mar = 2,
  xaxt = "n",
  yaxt = "n",
  axis.args = list(cex.axis = 0.5)
)
grid(nx = ncol(marks_mat), ny = nrow(marks_mat), col = "black", lwd = 0.5)
```




```
# Reset the figure setting
par(mfrow = c(1, 1))
```

Next, we apply DenMark main function to the prepared data.

```
# [Code Block 3.5] Run DenMark function

# Recall:
# grid_size: the number of grids per x/y- axis
# dataset: containing (x, y) coordinates and expression at the selected ROI
# basis: the number of basis functions for HSGP per x/y- axis
# boundfactor: (1, Inf) boundary factor to get rid of boundary effect in HSGP
# L1: half range in the x-axis
# L2: half range in the y-axis
# denmark_version: Choose from
#   (1) complete version DenMark (density-expression correlated) stan file, or
#   (2) simplified DenMark version (density-expression independent) stan file


# Run the full DenMark version:
denmark_output_full <- DenMarkmain(grid_size = grid_size,
                                   dataset = df_large_select,
                                   basis = basis_choice,
                                   boundfactor = boundary_choice,
                                   L1 = L1,
```

```r
                              L2 = L2,
                              denmark_version = denmark_stan)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 695.1 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 700.3 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 701.5 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 708.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 701.4 seconds.
## Total execution time: 708.8 seconds.
```

```r
# As well as the simplified version (for model comparison usage):
denmark_output_simple <- DenMarkmain(grid_size = grid_size,
                              dataset = df_large_select,
                              basis = basis_choice,
                              boundfactor = boundary_choice,
                              L1 = L1,
```

```r
                                L2 = L2,
                                denmark_version = denmark_simple_stan)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)

## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 726.8 seconds.
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 730.8 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 740.8 seconds.
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 741.6 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 735.0 seconds.
## Total execution time: 741.7 seconds.
```

Now let's check the fitted results.

```r
# [Code Block 3.8] Check the posterior summary

# 1. check posterior summary for these parameters:
```

```r
# beta0, beta1: intercepts for cell density/ gene expression latent field
# ell[1], ell[2]: length of scale for cell density/ gene expression latent field
# Astar[2,1]: the density-expression correlation.
# sigma[1], sigma[2]: SD of the cell density and gene expression latent field
# rhat: criteria to check if convergence
# ess_bulk and ess_tail: effective sample sizes
fit_summary_full <- denmark_output_full$fit$summary(variables = c("beta0",
                                                                  "beta1",
                                                        "ell[1]", 'ell[2]',
                                                       'Astar[2,1]',
                                                       'sigma[1]','sigma[2]'),
                                        c("mean","sd",
                                          "q2.5","q25","q50","q75","q97.5",
                                          "rhat","ess_bulk","ess_tail"))

# transfer to the data frame
summary_full<- denmark_output_full$fit$draws(format = "df")



fit_summary_full
```

```
## # A tibble: 7 x 11
##   variable  mean     sd `2.5%` `25%` `50%` `75%` `97.5%`  rhat ess_bulk ess_tail
##   <chr>    <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    4.11  0.686   2.68  3.68  4.11  4.56    5.38   1.00    1320.    1618.
## 2 beta1    0.967 0.131   0.672 0.890 0.975 1.06    1.20   1.00    1502.    1570.
## 3 ell[1]   0.825 0.0974  0.659 0.755 0.816 0.884   1.04   1.00    1235.    2008.
## 4 ell[2]   0.635 0.153   0.373 0.529 0.621 0.725   0.988  1.00    1041.    1762.
## 5 Astar[2~ 0.441 0.148   0.119 0.348 0.455 0.549   0.688  1.00    1442.    2021.
## 6 sigma[1] 2.77  0.280   2.30  2.57  2.74  2.93    3.39   1.00    1071.    1774.
## 7 sigma[2] 0.541 0.0716  0.427 0.490 0.533 0.582   0.707  1.00    1376.    2169.
```

```r
# 2. also check the posterior summary for the simplified DenMark version:
# beta0, beta1: intercepts for the cell density / gene expression latent field
# ell[1], ell[2]: length of scale for the cell density / gene expression latent field
# a11, a22: SD of the cell density and gene expression latent field
fit_summary_simple <- denmark_output_simple$fit$summary(variables = c("beta0",
                                                                      "beta1",
                                                        "ell[1]", 'ell[2]',
                                                       'a11', 'a22'),
                                        c("mean","sd",
                                          "q2.5","q25","q50","q75","q97.5",
                                          "rhat","ess_bulk","ess_tail"))
# transfer to the data frame
summary_simple<- denmark_output_simple$fit$draws(format = "df")


fit_summary_simple
```

```
## # A tibble: 6 x 11
##   variable  mean     sd `2.5%` `25%` `50%` `75%` `97.5%`  rhat ess_bulk ess_tail
```

```
##   <chr>    <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    4.17  0.536   3.11  3.82  4.17  4.51    5.23   1.00    3034.    2772.
## 2 beta1    1.03  0.134   0.744 0.951 1.04  1.12    1.26   1.00    3181.    2325.
## 3 ell[1]   0.760 0.0746  0.623 0.707 0.757 0.808   0.918  1.00    2402.    2768.
## 4 ell[2]   0.741 0.158   0.489 0.629 0.722 0.834   1.10   1.00    1530.    1995.
## 5 a11      2.38  0.182   2.04  2.25  2.37  2.50    2.77   1.00    2027.    2618.
## 6 a22      0.518 0.0723  0.406 0.468 0.507 0.555   0.681  1.00    1709.    2377.
```

```r
# [Code Block 3.9] Other fitting checks: WAIC and running time

# Show the WAIC:
print(paste0('The WAIC for the full DenMark model is: ',
             round(denmark_output_full$waic$waic, 2) ))
```

```
## [1] "The WAIC for the full DenMark model is: 1403.81"
```

```r
print(paste0('The WAIC for the simplified DenMark model is: ',
             round(denmark_output_simple$waic$waic, 2) ))
```

```
## [1] "The WAIC for the simplified DenMark model is: 1409.28"
```

```r
# Show the running time:
print(paste0('The running time for the full DenMark version is: ',
             round(denmark_output_full$time, 2),' mins.' ))
```

```
## [1] "The running time for the full DenMark version is: 11.81 mins."
```

```r
print(paste0('The running time for the simplified DenMark version is: ',
             round(denmark_output_simple$time,2), ' mins.' ))
```

```
## [1] "The running time for the simplified DenMark version is: 12.36 mins."
```

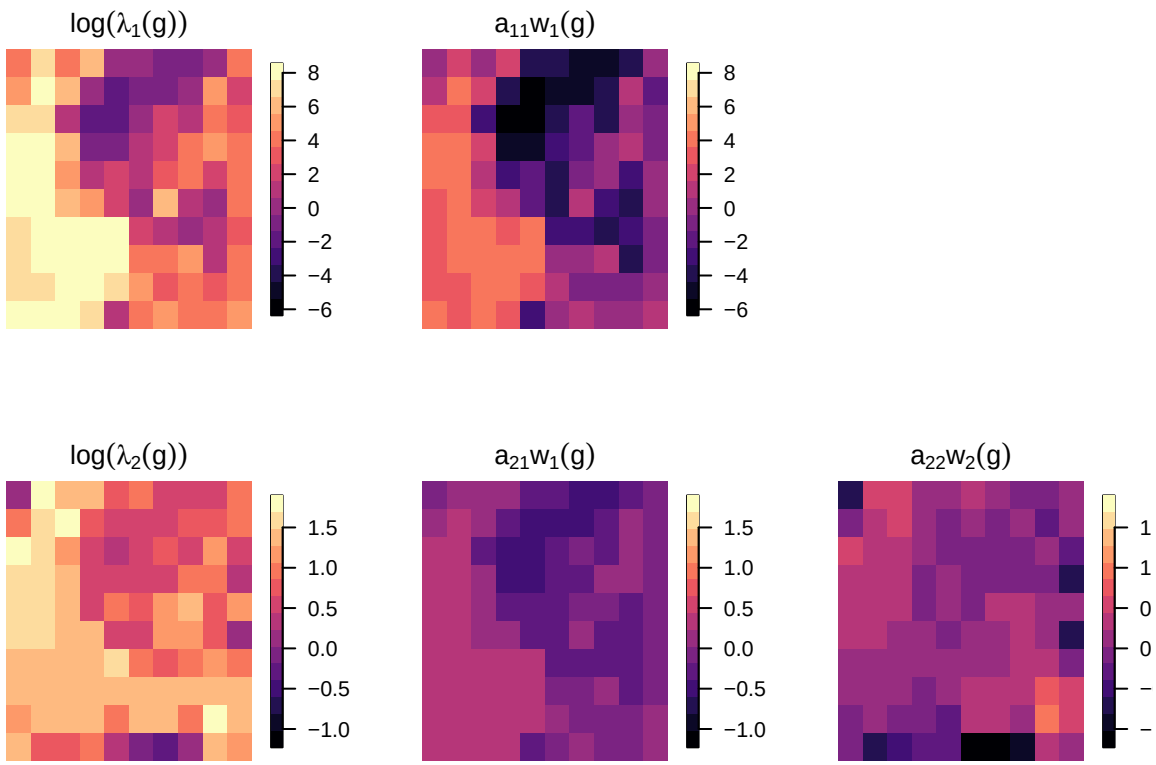
It is of interest to check the latent field decomposition for each grid. We visualize the decomposition for the fitted results under the full version DenMark.

```r
# [Code Block 3.10] Latent field decomposition

# summary of the posterior
samples_beta0<- summary_full[['beta0']]
samples_beta1<- summary_full[['beta1']]
samples_corr<- summary_full[['Astar[2,1]']]
samples_rho1<- summary_full[['ell[1]']]
samples_rho2<- summary_full[['ell[2]']]
samples_sigma1<- summary_full[['sigma[1]']]
samples_sigma2<- summary_full[['sigma[2]']]
# LMC matrix
samples_a11<- samples_sigma1
samples_a21<- samples_sigma2*samples_corr
samples_a22<- samples_sigma2*sqrt(1-samples_corr^2)
```

```r
# a11*w1= sigma1*w1
cal_a11w1<- function(i){
  w1<-denmark_output_full$fit$draws(variables = paste0("w1[",i, "]"))
  samples_a11*w1
}

# a21*w1 = a21_star*sigma2*w1
cal_a21w1<- function(i){
  w1<-denmark_output_full$fit$draws(variables = paste0("w1[",i, "]"))
  samples_a21*w1
}

# a22*w2= sqrt(1-a21.star^2)*sigma2 * w2
cal_a22w2<- function(i){
  w2<-denmark_output_full$fit$draws(variables = paste0("w2[",i, "]"))
  samples_a22*w2
}

# eta1 = a11*w1
cal_eta1<- cal_a11w1

# eta2 = a21*w1+a22*w2
cal_eta2<- function(i){
  w1<- denmark_output_full$fit$draws(variables = paste0("w1[",i, "]"))
  w2<- denmark_output_full$fit$draws(variables = paste0("w2[",i, "]"))
  (samples_a21*w1) + (samples_a22*w2)
}


# w1
cal_w1<- function(i){
  denmark_output_full$fit$draws(variables = paste0("w1[",i, "]"))
}
samples_w1<- sapply(1:(grid_size^2), cal_w1)

# w2
cal_w2<- function(i){
  denmark_output_full$fit$draws(variables = paste0("w2[",i, "]"))
}
samples_w2<- sapply(1:(grid_size^2), cal_w2)


# samples for the decomposition
samples_a11_w1<- sapply(1:(grid_size^2), cal_a11w1)
samples_a21_w1<- sapply(1:(grid_size^2), cal_a21w1)
samples_a22_w2<- sapply(1:(grid_size^2), cal_a22w2)
samples_eta1<- samples_a11_w1
samples_eta2<- sapply(1:(grid_size^2), cal_eta2)


# add legend bar to each of the plot:

# --- helper for rotation ---
```

```r
rotate_reflect <- function(m) {
  t(apply(m, 2, rev))
}

# --- compute matrices ---
mat1 <- rotate_reflect(apply(matrix(apply(samples_beta0 + samples_a11_w1, 2, mean),
                                    nrow = grid_size, byrow = TRUE), 2, rev))
mat2 <- rotate_reflect(apply(matrix(apply(samples_a11_w1, 2, mean),
                                    nrow = grid_size, byrow = TRUE), 2, rev))
mat3 <- rotate_reflect(apply(matrix(apply(samples_beta1 + samples_eta2, 2, mean),
                                    nrow = grid_size, byrow = TRUE), 2, rev))
mat4 <- rotate_reflect(apply(matrix(apply(samples_a21_w1, 2, mean),
                                    nrow = grid_size, byrow = TRUE), 2, rev))
mat5 <- rotate_reflect(apply(matrix(apply(samples_a22_w2, 2, mean),
                                    nrow = grid_size, byrow = TRUE), 2, rev))

# --- define row-specific scales ---
vals_row1 <- c(mat1, mat2)
breaks_row1 <- seq(min(vals_row1), max(vals_row1), length.out = 16)
cols_row1   <- viridis(length(breaks_row1) - 1, option = "magma")

vals_row2 <- c(mat3,mat4, mat5)
breaks_row2 <- seq(min(vals_row2), max(vals_row2), length.out = 16)
cols_row2   <- viridis(length(breaks_row2) - 1, option = "magma")


# --- layout: 2 rows (3 plots + 2 plots) ---

layout(matrix(c(1,2,3,
                4,5,6), nrow=2, byrow=TRUE))

par(mar = c(4, 4, 2, 2))

# Row 1: for cell density
image.plot(mat1, col = cols_row1, breaks = breaks_row1,
           main = expression(log(lambda[1](g))),
           xlab = '', ylab = '', axes = FALSE,
           legend.mar = 4,
           legend.cex = 0.8)

image.plot(mat2, col = cols_row1, breaks = breaks_row1,
           main = expression(a[11]*w[1](g)),
           xlab = '', ylab = '', axes = FALSE,
           legend.mar = 4,
           legend.cex = 0.8)

plot.new()  # empty slot

# Row 2: for gene expression (shared scale)
image.plot(mat3, col = cols_row2, breaks = breaks_row2,
      main = expression(log(lambda[2](g))),
      xlab = '', ylab = '', axes = FALSE,
      legend.mar = 4,
```

```
      legend.cex = 0.8)

image.plot(mat4, col = cols_row2, breaks = breaks_row2,
           main = expression(a[21]*w[1](g)),
           xlab = '', ylab = '', axes = FALSE,
           legend.mar = 4,
           legend.cex = 0.8)

image.plot(mat5, col = cols_row2, breaks = breaks_row2,
           main = expression(a[22]*w[2](g)),
           xlab = '', ylab = '', axes = FALSE,
           legend.mar = 4,
           legend.cex = 0.8)
```

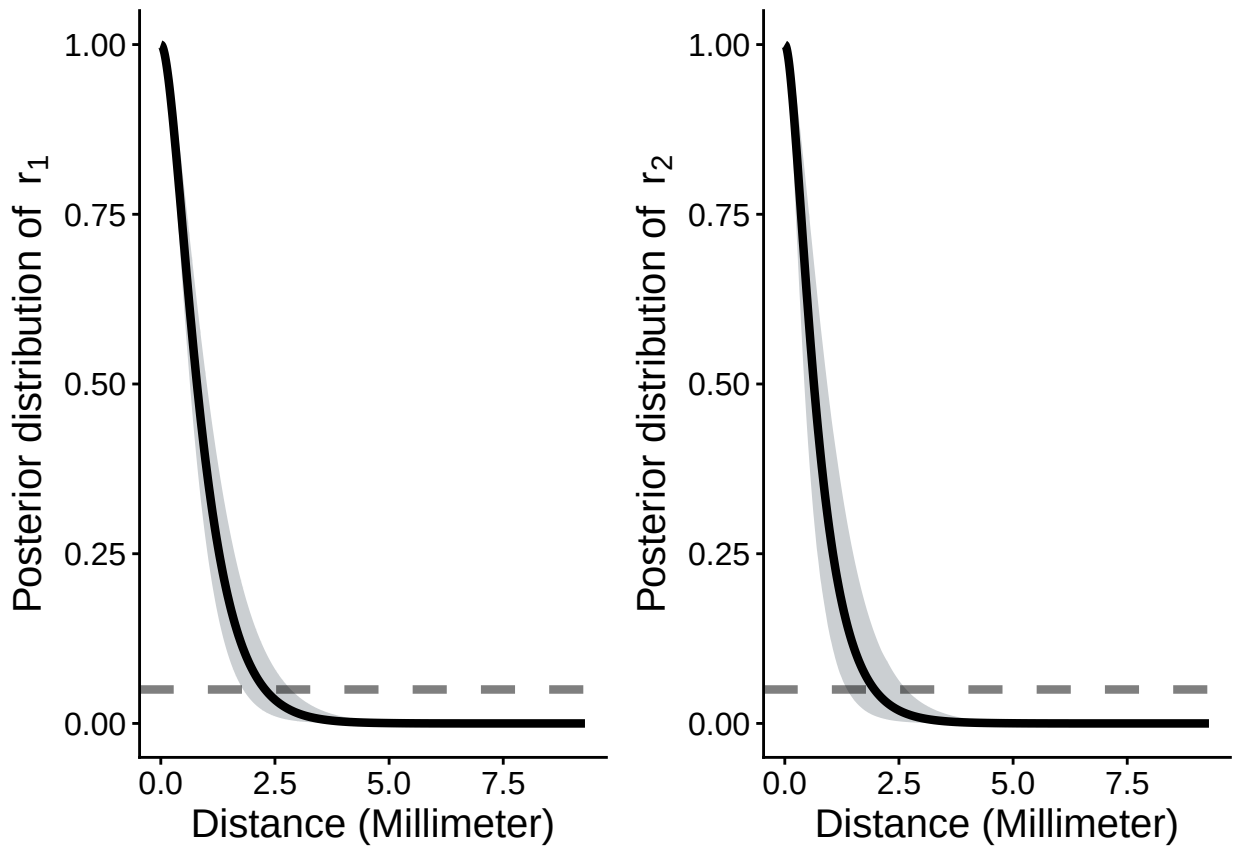


We might also want to check the spatial correlation functions for the two latent fields. This correlation measure how fast the two latent fields decay.

```
# [Code Block 3.11] Spatial correlation function

# a series of distance from (0, max(ROI distance)):
max_dist <- 2*sqrt(L1^2 + L2^2) # max distance of the ROI
dist_choice<- seq(0, max_dist, by=0.01)

# the samples from the DenMark full model
samples_rho1<- summary_full$`ell[1]`
samples_rho2<- summary_full$`ell[2]`
```

```r
samples_a21_HS<- summary_full$`sigma[2]`*summary_full$`Astar[2,1]`
samples_a22_HS<- summary_full$`sigma[2]`*sqrt(1- summary_full$`Astar[2,1]`^2)

# save the samples
post_rho1<- matrix(NA, nrow=length(dist_choice), ncol=length(samples_rho1))
post_rho2<- matrix(NA, nrow=length(dist_choice), ncol=length(samples_rho2))
post_mean_rho1<- rep(NA, length(dist_choice))
post_upper_rho1<- rep(NA, length(dist_choice))
post_lower_rho1<- rep(NA, length(dist_choice))
post_mean_rho2<- rep(NA, length(dist_choice))
post_upper_rho2<- rep(NA, length(dist_choice))
post_lower_rho2<- rep(NA, length(dist_choice))

for(k in 1:length(dist_choice)){
  post_rho1[k,]<- matern32(d=dist_choice[k], sigma=1, rho= samples_rho1)
  post_rho2[k,]<- (samples_a21^2*matern32(d=dist_choice[k],
                                          sigma=1,
                                          rho= samples_rho1)+
                     samples_a22^2*matern32(d=dist_choice[k],
                                            sigma=1,
                                            rho=samples_rho2))/(samples_a21^2+
                                                                samples_a22^2)
}

for (k in 1:length(dist_choice)){
  post_mean_rho1[k]<-mean(post_rho1[k,])
  post_upper_rho1[k]<-quantile(post_rho1[k,], 0.975)
  post_lower_rho1[k]<-quantile(post_rho1[k,], 0.025)

  post_mean_rho2[k]<-mean(post_rho2[k,])
  post_upper_rho2[k]<-quantile(post_rho2[k,], 0.975)
  post_lower_rho2[k]<-quantile(post_rho2[k,], 0.025)
}


# The latent field for cell density
df_rho1<- cbind( dist_choice ,
                 post_mean_rho1,
                 post_upper_rho1,
                 post_lower_rho1)

colnames(df_rho1)<- c('dist_choice',
                      'post_mean_rho1',
                      'post_upper_rho1',
                      'post_lower_rho1')

est_range_1<-dist_choice[which(abs(df_rho1[,2]-0.05)==min(abs(df_rho1[,2]-0.05)))]

eta1_corr<-ggplot(data=df_rho1, aes(x=dist_choice))+
  geom_ribbon(aes(ymin=post_lower_rho1, ymax= post_upper_rho1), fill = "#cbcfd2")+
  geom_line(aes(y=post_mean_rho1), color='#000', linewidth=1.5)+
  geom_hline(yintercept=0.05, linetype='dashed', col='#000', linewidth=1.5, alpha=.5)+
  xlim(0, 2*sqrt(L1^2 + L2^2))+
```

```r
  labs(x='Distance (Millimeter)', y=expression( 'Posterior distribution of '~ r[1]) )+
  theme_classic()+
  theme(axis.text.x = element_text(angle = -0, hjust = 0))+
  theme_classic() +
  theme(
    axis.text.x = element_text(size = 12),
    axis.text.y = element_text(size = 12),
    axis.title.x = element_text(size = 15),
    axis.title.y = element_text(size = 15),
    legend.position = "none"
  )


# The latent field for gen expression
df_rho2<- cbind( dist_choice,
                 post_mean_rho2,
                 post_upper_rho2,
                 post_lower_rho2)
colnames(df_rho2)<- c('dist_choice',
                      'post_mean_rho2',
                      'post_upper_rho2',
                      'post_lower_rho2')
est_range_2<-dist_choice[which(abs(df_rho2[,2]-0.05)==min(abs(df_rho2[,2]-0.05)))]

eta2_corr<- ggplot(data=df_rho2, aes(x=dist_choice))+
  geom_ribbon(aes(ymin=post_lower_rho2, ymax= post_upper_rho2), fill = "#cbcfd2")+
  geom_line(aes(y=post_mean_rho2), color='#000', linewidth=1.5)+
  xlim(0, 2*sqrt(L1^2 + L2^2))+
  geom_hline(yintercept=0.05, linetype='dashed', col='black', linewidth=1.5, alpha=.5)+
  labs(x='Distance (Millimeter)', y=expression( 'Posterior distribution of '~ r[2]))+
  theme_classic()+
  theme(axis.text.x = element_text(angle = -0, hjust = 0))+
  theme_classic() +
  theme(
    axis.text.x = element_text(size = 12),
    axis.text.y = element_text(size = 12),
    axis.title.x = element_text(size = 15),
    axis.title.y = element_text(size = 15),
    legend.position = "none"
  )

# show the figure:
plot_grid(eta1_corr, eta2_corr)
```




```r
# The estimated ranges:
print(paste0('The etimated range for cell density is: ',
             round(est_range_1, 2),' mm.' ))
```

```
## [1] "The etimated range for cell density is: 2.29 mm."
```

```r
print(paste0('The estimated range for gene expression is: ',
             round(est_range_2, 2), ' mm.' ))
```

```
## [1] "The estimated range for gene expression is: 1.96 mm."
```

4. DCG Discovery: Identify the density correlated genes in the dataset

This analysis can be finished by running each candidate gene and select the genes with

- PCG: positively density-expression ($Astar[2,1] > 0$) as positive density-correlated genes (PCG);
- NCG: negatively density-expression ($Astar[2,1] < 0$) as positive density-correlated genes (NCG).